

Feature: Cancel

From build-plan: feature 7a

Branch: feature/07a-cancel

Status: verified. Whole feature seen and agreed by Frank 2026-10-03; steps 7a.1 to 7a.6 built, tested and reviewed step by step; no P0 or P1 was ever open; decisions 10 and 11 made with Frank along the way. The checkpoint for the final review.

Goal

Jane's confirmation carries a private link. It opens a page, under the business's brand, showing her booking, with a Cancel button. Cancelling frees the time so someone else can book it, removes the event from the booked person's Google, tells Jane and the business by email (Jane's calendar invite is withdrawn too), and records it on her timeline. Nobody signs in: the link is the key. This is Phase 2's exit line "the cancel link in that email works". Moving a booking to another time is 7b, on the same page.

In scope

- The booking's private link: a token that names one booking and cannot be guessed or altered, and the public route that answers what the page shows. - Cancel: one public route, one transaction (the booking cancelled, its held time released, a `booking_cancelled` timeline entry), safe to press twice. - After the cancel, without Jane's answer waiting: the event removed from the booked person's Google, the cancellation email to Jane with her invite withdrawn, and the business's notification. - Jane's page in the frontend: her booking under the business's name, logo and colour, a Cancel button with a confirm step, and every state (not found, already cancelled, already started, failure). - The link in Jane's confirmation email.

Decisions made in the spec

1. **The link is the only key.** No sign-in and no account, like accepting a quote later (the project plan). Whoever holds the link can see and cancel that one booking, nothing else; every other booking, the customer's details and the business's data stay out of reach. 2. **A wrong, altered or unknown link answers exactly like a missing booking:** 404 `not_found`, nothing said about why. A link cannot be used to learn whether a booking exists. 3. **The page shows the booking, never the customer.** The service, the day and time in the business's zone, the person, the business's name, logo, colour and phone. Not Jane's name, email, phone, address or words: a forwarded email must not hand those on. 4. **Cancelling twice is one cancel.** The second press answers the same cancelled booking; no second timeline entry, no second email (each email's idempotency key is the booking id and its kind). 5. **A cancel never waits for Google or Resend** (feature 6's decision 1, 5d's decision 6). The booking is cancelled and its time free the moment the answer goes back; a failed event removal or email logs one line, and feature 8 retries. 6. **The time is free at once.** Its held rows are cancelled in the same transaction, so the free-times route offers it again straight away. The rows are kept, as feature 5a decided. 7. **The lead stays where it is.** A cancel does not move the lead to another stage or close it; the pipeline (feature 14) decides what a cancel does to a lead. The timeline entry is what the owner sees. 8. **Jane's invite is withdrawn, not left behind.** Her cancellation email carries an `invite.ics` with `METHOD:CANCEL`, the same UID and a higher `SEQUENCE`,

so her calendar removes the event instead of keeping a booking that no longer exists. 9. **Jane's page names the business, never the product.** Its title, icon and text come from the business; nothing on it says the product's name. The domain the app runs on must not either (a deploy note, below). 10. **The link is signed, nothing stored** (Frank, 2026-10-03, open question 1). The token is the booking id and an HMAC-SHA256 of it under a new key, `BOOKING_LINK_KEY`. A stolen copy of the database opens no booking; every later email (7b's moved email, feature 8's reminder) can carry the same link; changing the key ends every link at once. Rejected: a random token stored on the booking (the overview's `cancelToken`), where a database copy opens every booking; a stored hash of a random token, safe at rest but no later email could carry the link again. 11. **Jane may cancel until the appointment starts** (Frank, 2026-10-03, open question 2). From its start the page shows the business's phone instead: once the worker is on site it is a conversation, not a button. A late cancel still frees the time. A business's own cutoff ("not within 24 hours") comes with Settings, feature 12. Rejected: a 24-hour cutoff for every business, a rule nobody chose.

Out of scope

- Moving a booking: 7b, on the same page and the same link.
- The owner cancelling from the dashboard: features 11 and 12b, which can call the same cancel function.
- A business's own cancellation rule (for example "not within 24 hours"): Settings, feature 12. Until then it is decision 11: until the start.
- Retrying a failed event removal or email: feature 8.
- What a cancel does to the lead's stage: feature 14.
- Rate limits on the public routes: feature 9.
- A text to the customer with the link: feature 8; a phone-only booking has no email, so it gets no link yet.

Build loop

Steps are built one at a time on `feature/07a-cancel`. Each step's plan gets Frank's yes just before it is built. After that yes nothing stops until the review: build, tests, tick the box, the build log entry, commit with the step number and push to the feature branch, `/audit` scoped to the step, then the independent review (`workflow.stepReview: "every"`, `workflow.checkpointCommits: "enabled"`). Findings are talked through after the review; P0/P1 are fixed before the next step. `/complete` makes the merge commit, on Frank's yes.

No package is planned. Installing one is a line only Frank crosses.

Build steps

- [x] **7a.1 The booking's private link.** - The token is signed, nothing stored (decision 10): the booking id and an HMAC-SHA256 of it under a new key `BOOKING_LINK_KEY` (32 random bytes, base64), compared in constant time.
- `backend/lib/booking/booking-page-token.ts`:
`makeBookingPageToken(bookingId)` and `readBookingPageToken(token)` (the booking id, or null for anything altered, truncated or made up). The key is read with the other settings at start: production refuses to start without it, as without `CALENDAR_TOKEN_KEY`; `.env.example` names it.
- `backend/lib/booking/find-booking-page.ts` and `GET /public/bookings/:token` (public CORS, never with credentials, `Cache-Control: no-store`): the page's view of one booking (Data / contracts), read through the booking's own business. Unknown, altered or another business's: 404 `not_found` (decision 2).
- **Done when** saved tests: a made token reads back to its booking; one changed character, a

cut-off token, another booking's signature or a made-up string reads as null; the route answers the view for a real link and 404 for every bad one, with the same body; the view carries none of the customer's details; production without the key refuses to start; the answer is never cached. The frontend build passes and the typed public client sees the route.

- [x] **7a.2 Cancel.** - Jane may cancel until the appointment starts (decision 11). The page's view gains `canCancel` : confirmed and not started. - `backend/lib/booking/cancel-booking.ts` : in one transaction, the booking row locked, then, when it is confirmed and not started: its status `cancelled` , its held rows released (`releaseTime`), and a `booking_cancelled` timeline entry on the contact. Already cancelled: answers that, writes nothing (decision 4). Started: refuses, writes nothing. - `booking_cancelled` joins the activity types in `packages/shared` , and migration 0017 regenerates the timeline's type check. - `POST /public/bookings/:token/cancel` : answers the page's view with status `cancelled` ; 404 for a bad link; 409 `already_started` once the appointment has started. - **Done when** saved tests, against the local database: a cancel marks the booking cancelled, releases its rows and writes one timeline entry; the free-times route offers the time again; a second cancel answers the same and writes nothing more; a started booking is refused and nothing changes; two cancels at the same instant make one cancel; another business's booking is never touched; nothing in the answer, the log or the timeline payload carries the customer's details.

- [x] **7a.3 The event leaves the booked person's Google.** - The provider seam gains `deleteEvent(accessToken, eventId)` : Google's `DELETE` , an event already gone (404, 410) being the answer too; throws on any other failure. `backend/lib/calendar/remove-booking-event.ts` removes a cancelled booking's event by the id made from the booking, saved or not, and clears its `calendarEventId` ; no connection does nothing, and a person with a connection but no event gets a `DELETE` that Google answers 404, which counts as done. - Started by the cancel after its transaction, not awaited, beside the Google write and in its shape (decision 5); the removals still running can be awaited by tests. - **Done when** saved tests, with Google faked: a cancel removes the event and clears its id; an event already gone (404 or 410) counts as removed, with no warning; no connection makes no call; the `DELETE` carries the person's own access key; a Google error keeps the cancel, logs one line with no customer details and leaves the id; the cancel's answer does not wait for Google.

- [x] **7a.4 Both are told.** - `bookingIcs` gains the cancelling form: `METHOD:CANCEL` , `STATUS:CANCELLED` , the same UID, `SEQUENCE:1` (decision 8). - `backend/emails/booking-cancelled.tsx` to Jane: "Your booking is cancelled", the service and the time it was, the business's phone, never the product's name; the withdrawn invite attached (`text/calendar; charset=utf-8; method=CANCEL`).

`backend/emails/booking-cancelled-notification.tsx` to the business: "Cancelled: <service>, <day and time>", the customer's name, phone and email, Reply to the customer. - Sent after the cancel, not awaited, in the shape of the confirmation emails: Jane's when she gave an email, the business's always (the customer cancelled, so the business needs to hear even for a booking the owner made). Keys `booking-cancelled/<id>` and `booking-cancelled-notification/<id>` . Each email that went gets an `email_sent` timeline entry. Decision 4 of feature 6 holds: a business without its addresses, key or time zone sends nothing and logs it. - `npm run email:preview` writes the two new emails as well. - **Done when** saved tests, with Resend faked: a cancel sends both, the cancelling invite attached to Jane's

with the booking's UID; a phone-only booking sends only the business's; a second cancel sends nothing more; Resend failing keeps the cancel and logs one line; the templates render the time in the business's zone, no product name, the customer's text as text; the preview writes four files, the two new ones opened by hand, the screenshots in the log.

- [x] **7a.5 Jane's page.** - `frontend/app/b/[token]/page.tsx` : the booking under the business's name, logo and colour; the service, the day and time with the zone named, the person; a Cancel button that asks once more ("Cancel this booking?") before it sends, locks while sending, and ends on "Your booking is cancelled". States: not found ("This link doesn't work. Call the business."), already cancelled, already started (with the business's phone), and an unexpected failure that keeps the button usable. Called through the typed public client, never with the login cookie. - The page names the business and never the product (decision 9): its own title and no product icon; `noindex`, `Referrer-Policy: no-referrer` so the link never leaks to another site, and never cached. - Accessible: the confirm step and the result are announced, focus moves to the result, the button has a clear name. - **Done when** the frontend build and lint pass; checked by hand in the browser on a dev booking: the page as Jane sees it, the confirm step, the cancelled page, a bad link, a booking already started; the page's title and icon carry no product name; the screenshots in the log.

- [x] **7a.6 The link in Jane's confirmation.** - The confirmation email gains "Need to change it? Manage your booking", linking `<APP_ORIGIN>/b/<token>`, in the HTML and the plain-text twin. Only the customer's email carries it; the business's notification never does. - **Done when** saved tests: the confirmation carries a link that opens the same booking's page (its token reads back to that booking); the notification carries none; the preview shows it, opened by hand.

Files / areas

- `backend/lib/booking/` : `booking-page-token.ts`, `find-booking-page.ts`, `cancel-booking.ts`, the after-cancel starts beside `booking-event-writes.ts` and `booking-confirmation-emails.ts` - `backend/lib/calendar/` : `calendar-provider.ts`, `google-calendar-provider.ts` (`deleteEvent`), `remove-booking-event.ts` - `backend/lib/email/` : `booking-ics.ts`, the sending of the two new emails, `find-booking-email-recipients.ts` if the cancel rule lives there - `backend/emails/` : `booking-cancelled.tsx`, `booking-cancelled-notification.tsx`, `booking-confirmation.tsx` (the link), the preview script - `backend/routes/public-bookings-routes.ts` (or a new `public-booking-page-routes.ts`), `backend/app.ts`, `backend/server.ts` and the settings reader (the key) - `packages/shared/crm/activity-types.ts`, migration 0017 - `frontend/app/b/[token]/`, `frontend/lib/api-client.ts` - `.env.example` - their tests

Data / contracts

The token (7a.1, decision 10): `<booking id>.<signature>`, the signature `base64url(HMAC-SHA256(BOOKING_LINK_KEY, "booking-page:" + bookingId))`, 43 characters. Read back only when it is exactly that shape and the signature matches in constant time. Never logged, never stored.

The page's view (`GET /public/bookings/:token`, 200):

```

type BookingPageType = {
  status: "confirmed" | "cancelled";
  canCancel: boolean; // confirmed and not started (decision 11); added in 7a.2
  service: string;
  startsAt: string; // ISO 8601 UTC
  endsAt: string;
  timezone: string; // the business's IANA zone
  person: string; // the booked person's name
  business: {
    name: string;
    logo: string | null;
    brandColor: string | null;
    phone: string | null;
    website: string | null;
  };
};

```

No customer details, no ids but what the link already carries. 404 `not_found` for every bad link, the same body each time.

Cancel (`POST /public/bookings/:token/cancel`): 200 the view with `status: "cancelled"` ; 404 `not_found` ; 409 `already_started` . Safe to repeat.

The timeline entry (7a.2): `type: "booking_cancelled"` , payload `{ bookingId }` , `actorUserId` null (the customer did it).

The withdrawn invite (7a.4): `invite.ics` , `text/calendar`; `charset=utf-8`; `method=CANCEL` ; `METHOD:CANCEL` , `STATUS:CANCELLED` , the booking's UID, `SEQUENCE:1` , the original times. Stamped (`DTSTAMP`) with the moment of the cancel, read from its timeline entry, so a retry sends the very same invite.

The cancellation emails (7a.4): keys `booking-cancelled/<id>` and `booking-cancelled-notification/<id>` ; each that went gets an `email_sent` entry with `kind booking_cancellation` or `booking_cancellation_notification` (feature 6's two kinds keep theirs).

The page (7a.5): `/b/<token>` on the frontend's origin (`APP_ORIGIN`).

Testing

Backend Vitest against the local database, as feature 6: businesses of the tests' own, removed after; Google and Resend faked, no test reaches either. The token functions are pure. Each case on the feature's Simulate page is a saved test under the same name, in the step that builds its rule. The page is checked by hand in the browser (there is no browser test harness), with screenshots in the log.

Notes for the AI

- The booking's id is a `randomUUID()` ; the token adds the signature, so the id alone opens nothing. - Read the booking through its own business: the token gives the booking id, the booking row gives its business, and every other read uses that business. - `releaseTime` takes commitment ids: select the booking's own active rows first, inside the transaction. - Lock the booking row (`for update`) before deciding, so two cancels at once make one. - The Google event's id is the booking id without dashes; removing it must treat 404 and 410 as done. - The root layout's title says the product's name: the page overrides it. - Never put a token, a customer's details or a key in a log line. Railway's and Vercel's own access logs do record request paths, the link included: that comes with the link being the key (decision 10), and is named at deploy. - From step 7a.4's review: the invite's SEQUENCE is fixed at 0 (made) and 1 (cancelled); 7b needs it to count changes, so a moved booking sends a higher one and a cancel after a move higher again. The business's cancellation notice says the customer cancelled from their link; the owner's own cancel (features 11 and 12b) needs its own wording. F-128: no real calendar has yet been shown to remove the event from the attached CANCEL invite; checking it means a real email, Frank's call. - From step 7a.3: the event is removed by the id made from the booking, so a cancel before its id was saved still removes it. One gap is left: a cancel in the first moment after booking, whose Google write finishes after the removal, leaves the event in place. Feature 8's retries close it. - From step 7a.3's review, for 7b: Google keeps a deleted event's id, and a new event with the same id is refused (409), which `createEvent` reads as "already there". So 7b moves an event by updating it in place, never by deleting and writing it again. - From step 7a.2's review: 7a.3 and 7a.4 start Google and the emails only when the cancel changed something (`alreadyCancelled` false), so a second press never removes or emails twice. `cancelBooking` fixes the actor to the customer and refuses after the start; the owner's screens (features 11 and 12b) will need their own actor, and may need to cancel after the start. - From step 7a.1's review: a business may have the slug `bookings` , which sits where `/public/bookings/...` does. Nothing collides today; reserve the slug before 7b adds more routes there. 7a.2 adds a test that a cancelled booking's page still opens. - Deploy note: `BOOKING_LINK_KEY` on Railway; migration 0017; the domain in `APP_ORIGIN` is what Jane sees in her link, so it must not carry the product's name.

Open questions

Each blocks only the step named; Frank answers it when that step's plan is gone through.

1. Answered 2026-10-03: decision 10. 2. Answered 2026-10-03: decision 11.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `crm/activity-types.ts` : `booking_cancelled` joins the timeline's types; migration `0017_booking_cancelled_activity.sql` , generated and untouched, widens the type check. - `crypto/token-cipher.ts` : `readBase64Key(name, value)` , the one reader for every 32-byte key the API holds; the calendar lock and the booking link key both go through it, so each error names its own setting. - `helpers/format-booking-time.ts` : moved from the backend into the shared package,

because the customer's page shows the same "Thursday, October 8 at 9:00 a.m. MDT" as the emails. - `helpers/text-color-on.ts` (7a.5's review, F-130): white or dark ink by contrast on the business's colour, used by the page's buttons and the emails' button.

backend: the private link

- `lib/booking/booking-page-token.ts` : `<booking id>.<signature>` , an HMAC-SHA256 of `booking-page:<id>` under `BOOKING_LINK_KEY` (decision 10). Nothing is stored; only one spelling of a signature opens; compared in constant time. `read-booking-link-key.ts` reads the key, and `server.ts` calls it at start, so production refuses to run without it. - `lib/booking/find-booking-page.ts` : the page's view of one booking, read through the booking's own business: no customer details, no ids beyond the link's own (decision 3). - `routes/public-booking-page-routes.ts` : `GET /public/bookings/:token` and `POST /public/bookings/:token/cancel` , public CORS without credentials, `no-store` . Every bad link gets the same 404 (decision 2). - `lib/booking/booking-page-url.ts` (7a.6): `<APP_ORIGIN>/b/<token>` , the address the confirmation links. The same booking always gives the same address, so a retried email is byte-identical under the same Resend key.

backend: the cancel

- `lib/booking/cancel-booking.ts` : one transaction, the booking row locked first, then, only when it is confirmed and not started: status `cancelled` , its held rows released, one `booking_cancelled` entry with no actor (the customer did it). Already cancelled answers the same and writes nothing (decision 4); started refuses with `already_started` (decision 11). The Google removal and the emails start only when this call changed something, so a second press does neither again. - `lib/booking/booking-event-removals.ts` and `booking-cancellation-emails.ts` : started after the transaction, never awaited (decision 5), in the shape of feature 5's event writes and feature 6's confirmation sends; a test can await them.

backend: Google

- `lib/calendar/calendar-event-id-of.ts` : the event's id made from the booking's id, now shared by the write and the removal, so removing needs nothing stored and works even before the id was saved. - `google-calendar-provider.ts` gains `deleteEvent` ; 404 and 410 count as done. `remove-booking-event.ts` removes the event with the booked person's own access key and clears `calendarEventId` ; no connection makes no call.

backend: the emails

- `lib/email/booking-ics.ts` : the cancelling form, `METHOD:CANCEL` , `STATUS:CANCELLED` , the same UID, `SEQUENCE:1` (decision 8). Stamped with the cancel's own timeline moment, so a retry sends the same invite. - `emails/booking-cancelled.tsx` (Jane: "Your booking is cancelled", the withdrawn invite attached) and `booking-cancelled-notification.tsx` (the business: who cancelled, Reply to the customer). The business is always told; Jane only when she gave an email. - `lib/email/find-booking-email-context.ts` , `send-and-record-emails.ts` and `log-nothing-sent.ts` : feature 6's reading and sending moved into shared files, so the confirmation and the cancellation use one path. The cancellation sender refuses a booking still confirmed; the confirmation sender refuses one already cancelled, so a late confirmation never follows a cancel. - `emails/booking-confirmation.tsx` (7a.6): "Need to change it?"

and a "Manage your booking" button in the business's colour, the address being its own argument so no other email can carry it; the phone became an "Or call us at ..." line under it.

frontend

- `app/b/[token]/page.tsx`, `app/b/layout.tsx` and `app/b/icon.svg`: the customer's page under the business's name; the tab says "Your booking with the business", never the product (decision 9); a neutral icon; `noindex`. - `next.config.ts`: `/b/*` answers with `Referrer-Policy: no-referrer`, `Cache-Control: no-store` and `X-Robots-Tag: noindex`. - `components/booking-page/booking-page.tsx`: the booking, Cancel with a confirm step ("Cancel this booking?", focus on "Keep it"), the result read out and focused, and every other state in plain words: a bad link, already cancelled, already started (the business's phone instead), and "can't load right now" with a "Try again" that shows it is working and keeps focus (F-131, F-133). Through the typed public client, never with the login cookie.

Ports

Mid-feature (`ae16f3c`), `scheduleads` moved to its own dev ports, 3400 for the frontend and 3401 for the API, so Frank's other projects on 3000/3001 are never stopped for a check. The frontend is pinned with `- p 3400` and fails loudly instead of drifting. AGENTS.md says how to find every place the ports live. Google's OAuth client needs `http://localhost:3401/calendar/callback` added by Frank.

Packages

None installed.

Tests

Every case on the feature's Simulate page is a saved test under the same name. Google and Resend are faked in every test that could reach them. The customer's page has no saved test, because the frontend has no test runner yet; it was checked by hand in the browser at phone width, with screenshots in the build log. Final count: 503 backend and 107 shared tests.

Review history

Every step was reviewed by a fresh reviewer before the next began; no P0 or P1 was ever open. The findings of each review (F-117 to F-133) were fixed before the next step, and the final review below passed over the whole feature, closed the three still marked fixed, and raised one P3, F-134, carried forward.

Carried forward

- F-134 (P3): the log half of one cancel test reads the log before the background work that writes it; those lines are tested elsewhere. - F-128 (unverified): no real calendar has yet been shown to remove the event from the cancelling invite; it needs a real email, Frank's call. - At deploy: `BOOKING_LINK_KEY` on Railway; migration 0017; the domain in `APP_ORIGIN` is what customers see in their link, so it must not carry the product's name. Railway's and Vercel's access logs record request paths, the link included. - Feature 8: retries for a failed removal or email, and the race where a cancel in the first moment after booking finishes before the Google write. Changing `BOOKING_LINK_KEY` or `APP_ORIGIN` between a send and its retry would change the email under the same key. - 7b: move the event by updating it in place (Google refuses a deleted id); the invite's SEQUENCE must count changes; reserve the slug `bookings` before adding

routes under `/public/bookings` . - Features 11 and 12b: the owner's own cancel needs its own actor and wording.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live. F-32, F-47, F-58, F-62, F-94, F-95, F-116, F-128, F-134 stay in the live ledger.

7a/F-117 [P2] closed - No test notices the booking page reading another business's time zone, or a person's hours row

File: backend/lib/booking/find-booking-page.ts:65-66 (tests: backend/routes/public-booking-page-routes.test.ts:64, :172) **Found:** 2026-10-03 by independent review of step 7a.1 (scope: 32114fc..4c05007; lenses: quality, security, performance, tests) **Why it matters:** The hours row that gives the page its zone is the one join not pinned by a foreign key to the booking's business, so its two conditions are what keep it inside the business. Both were removed in turn in this review and all 11 tests still passed: without the business condition the query picks any business's hours row, and without `isNull(resourceId)` a person's own row (zone always null) can be picked and a real link answers 404. The tests cannot see either because both test businesses share America/Edmonton, have no person-level hours, and the isolation test compares only the business name. The code is right today; the guard the spec asks for ("read through the booking's own business") is untested. **Suggested fix:** Give the second test business a different zone (for example America/Toronto) and a person-level hours row for Marco, and assert each link's `timezone` (and that the booking still opens) in "each link opens its own business's booking only". **Resolution:** Fixed 2026-10-03: the second test business runs on Toronto time and Marco gets a person-level hours row (no zone) after his booking; a new test expects each link to say its own business's zone. The lookup now refuses more than one joined row instead of taking the first, so a join that loses either condition fails deterministically: dropping the business condition or the person-row condition each fails three tests. Closed 2026-10-03 by independent review of step 7a.2 (scope: 2e91e73..6b0b747): re-examined find-booking-page.ts and its route tests. Dropping the hours row's business condition, or its person-row condition, each fails six route tests (the lookup refuses a second joined row); the partial unique index keeps one business-level hours row per business, so the refusal can never fire on good data. No new defect.

7a/F-118 [P3] closed - The page's status is typed as any string, not the contract's two values

File: backend/lib/booking/find-booking-page.ts:18 **Found:** 2026-10-03 by independent review of step 7a.1 (scope: 32114fc..4c05007; lenses: quality, security, performance, tests) **Why it matters:** The spec's view says `status: "confirmed" | "cancelled"` and the database check allows only those, but `BookingPageType.status` is `string`, so the frontend's inferred `BookingPageType` is `string` too. Jane's page (7a.5) branches on this value for its states, and with `string` a misspelt branch or a third status added later compiles without complaint, which is what the typed client exists to catch. **Suggested fix:** Type it as `"confirmed" | "cancelled"` (one shared union beside the booking table if other code needs it) and narrow the row value once in `findBookingPage`. **Resolution:** Fixed 2026-10-03: `BookingStatusType` is `"confirmed" | "cancelled"`, narrowed once in `findBookingPage`; any other value is refused, never shown. The typed client carries the union. Closed 2026-10-03 by independent review of

step 7a.2 (scope: 2e91e73..6b0b747): BookingStatusType is the two-value union, narrowed once in statusOf, and 7a.2's canCancel builds on the narrowed value. No new defect.

7a/F-119 [P3] closed - The token's exact construction is not pinned by a known-answer test

File: backend/lib/booking/booking-page-token.ts:13-14 (tests: backend/lib/booking/booking-page-token.test.ts:15) **Found:** 2026-10-03 by independent review of step 7a.1 (scope: 32114fc..4c05007; lenses: quality, security, performance, tests) **Why it matters:** Every test makes and reads a token with the same code, so they pass for any construction. Dropping the `booking-page:` purpose prefix in this review left all 11 tests green. Decision 10 makes these links permanent (every later email carries the same one), so an innocent change to the construction would silently end every link already sent, and the domain separation the comment promises is unguarded. **Suggested fix:** Add one test with a fixed key and a fixed booking id that expects the exact token string, computed once from the spec's formula `(base64url(HMAC-SHA256(key, "booking-page:" + id)))`. **Resolution:** Fixed 2026-10-03: a known-answer test pins the exact token for a fixed key and booking id; dropping the purpose prefix now fails it. Closed 2026-10-03 by independent review of step 7a.2 (scope: 2e91e73..6b0b747): re-examined booking-page-token.ts and its test; dropping the booking-page: prefix fails "a link is made exactly the same way, always". No new defect.

7a/F-120 [P3] closed - readBookingLinkKey is a line-for-line copy of readTokenKey

File: backend/lib/booking/read-booking-link-key.ts:7 (copy of packages/shared/crypto/token-cipher.ts:14) **Found:** 2026-10-03 by independent review of step 7a.1 (scope: 32114fc..4c05007; lenses: quality, security, performance, tests) **Why it matters:** The two functions differ only in the variable name in the env lookup and messages: the same canonical base64 check, the same length check, the same make-one hint. The canonical check is the subtle part (Buffer skips bad characters silently), and a fix to one copy will not reach the other; feature 16's quote link would make a third. **Suggested fix:** One shared `readBase64Key(name, encodedKey)` in `packages/shared/crypto` (or `helpers/`), with `readTokenKey` and `readBookingLinkKey` as one-line callers; the existing tests keep covering both names. **Resolution:** Fixed 2026-10-03: `readBase64Key(name, value)` in `packages/shared/crypto/token-cipher.ts` holds the check once; `readTokenKey` and `readBookingLinkKey` are one-line callers with their own names in the errors. Closed 2026-10-03 by independent review of step 7a.2 (scope: 2e91e73..6b0b747): `read-booking-link-key.ts` and `readTokenKey` are one-line callers of `readBase64Key` in `packages/shared/crypto/token-cipher.ts`; both names stay tested (`booking-page-token.test.ts`, `token-cipher.test.ts`). No new defect.

7a/F-121 [P3] closed - Two planning docs still contradict decision 10 and the new route

File: blueprint/context/coding-standards.md:209; blueprint/context/project-overview.md:149 **Found:** 2026-10-03 by independent review of step 7a.1 (scope: 32114fc..4c05007; lenses: quality, security, performance, tests) **Why it matters:** The standards' public-route rule says a public route is read-only, takes its business from the slug, and never finds a row by its id alone; `GET /public/bookings/:token` finds the booking by the signed id, which is right by the spec, and 7a.2's cancel will write. A later reviewer following the standard would flag correct code, or a builder would copy the slug rule onto the token route. The overview's planned booking model still lists `cancelToken`, the stored token decision 10 rejected; this commit edited the overview but left that line. **Suggested fix:** Add a second public-route kind to the

standard: keyed by a signed link, the business taken from the row the link names, every other read inside that business, every bad link the same 404. Drop `cancelToken` from the overview's booking line.

Resolution: Fixed 2026-10-03: coding-standards.md gains the signed-link kind of public route (its rules: the id only from a verified signature, every other query on that row's business, one 404, no customer details, never cached, the link never logged or stored); the build log's Rules tab regenerated. The overview drops `cancelToken` and names the signed link. Closed 2026-10-03 by independent review of step 7a.2 (scope: 2e91e73..6b0b747): coding-standards.md:215-222 names the signed-link public route and 7a.2's POST follows it (id only from the verified token, every other query on the row's business, one 404, no-store, no customer details); project-overview.md:150 names the signed link and `cancelToken` is gone. No new defect.

7a/F-122 [P3] closed - Neither guard against a double cancel, nor the cancelled-first order, is pinned by a test

File: backend/lib/booking/cancel-booking.ts:41, :44, :59 (tests: backend/lib/booking/cancel-booking.test.ts:176, :193) **Found:** 2026-10-03 by independent review of step 7a.2 (scope: 2e91e73..6b0b747; lenses: quality, security, performance, tests) **Why it matters:** The cancel has two guards against two presses at once, the row lock (:41) and the update that only changes a confirmed booking (:59). Removing either one alone left all 19 tests green; only removing both failed the race test. "Two cancels at the same instant" (:176) also passed with both guards removed, so its two calls never overlap and it repeats the second-cancel test rather than proving the Done when line it is named for. Removing the already-cancelled check (:44) also stays green, yet it is what makes a second press after the start answer 200 cancelled rather than 409 already_started. The code is right today; the lock is the guard 7b's move will lean on (a move and a cancel of the same booking at once), and nothing would notice it going. **Suggested fix:** In the race test, let the holding transaction move the booking into the past instead of cancelling it, and expect `already_started` with nothing changed: that fails without the lock and passes with it. Drop or rename the Promise.all test. Add one case: a cancelled booking whose start has passed answers already cancelled. **Resolution:** Fixed 2026-10-03: the Promise.all test, whose two calls never overlapped, is gone. A shared helper now holds the booking in another transaction while the cancel starts: holding and cancelling it gives "already cancelled" with no entry; holding and moving it into the past gives already_started, which fails without the row lock. A cancelled booking whose start has passed answers already cancelled, which fails without the cancelled-first check. The confirmed-only update stays a second guard that matters only without the lock: removing it alone passes, removing it with the lock fails the race test. Closed 2026-10-03 by independent review of step 7a.3 (scope: 12e7fd3..7df5944): re-run by mutation against the 7a.3 code, removing the row lock (cancel-booking.ts:45) fails "moved into the past meanwhile, it is refused"; removing the cancelled-first check (:48) fails "a cancelled booking whose start has passed still answers already cancelled"; removing the confirmed-only condition (:63) alone passes and with the lock fails both race tests, as the fix said. 7a.3's change to this file (the removal started after the transaction) leaves the guards as they were. No new defect.

7a/F-123 [P3] closed - The 404 answer, the one the agreed "remove by the made id" call relies on, is not tested

File: backend/lib/calendar/google-calendar-provider.ts:97 (tests: backend/lib/calendar/remove-booking-event.test.ts:179, :257) **Found:** 2026-10-03 by independent review of step 7a.3 (scope: 12e7fd3..7df5944; lenses: quality, security, performance, tests) **Why it matters:** The Done when says an event already gone counts as removed, and the spec names both 404 and 410. Only 410 is tested (:184). Deleting `response.status === 404` from :97 leaves all 30 tests in the three files green. 404 is what Google answers for an id it never had, which is exactly the case the agreed call creates: a connected person whose event was never written (connected after the booking, or the write failed) now gets a DELETE, and without the 404 rule every such cancel would log a false "is still there" line and keep feature 8 retrying forever. The "id not saved yet" test (:257) answers 204, so it does not cover it either. Smaller, same place: nothing checks the DELETE carries the person's own access token (dropping it from :93 also stays green). **Suggested fix:** Answer 404 in the "id not saved yet" test (that is what Google says when the write never happened) and expect no warning, or add a 404 case beside the 410 one. Record the Authorization header in the fake and expect the saved access token on the DELETE. **Resolution:** Fixed 2026-10-03: the "id not saved yet" test now has Google answer 404 and expects no warning; the fake records the Authorization header and the first test expects Ana's own saved key. Dropping the 404 rule, or sending another key, each fail a test. Closed 2026-10-03 by independent review of step 7a.4 (bea19bc..dfa2d60): re-run, removing `response.status === 404` from google-calendar-provider.ts fails "an event whose id was not saved yet is removed by its own id, and Google's 404 is done"; the fake records the Authorization header and the test expects the saved key. No new defect.

7a/F-124 [P3] closed - The 7a.3 step and its Done when still say "no event makes no call", which the agreed call made untrue

File: blueprint/context/current-feature.md:155, :161 **Found:** 2026-10-03 by independent review of step 7a.3 (scope: 12e7fd3..7df5944; lenses: quality, security, performance, tests) **Why it matters:** The step says "no event or no connection does nothing" (:155) and the Done when "no connection or no event makes no call" (:161). After the agreed call, a booking with no event but a connected person does make a DELETE (remove-booking-event.ts:31, proved by the test at remove-booking-event.test.ts:257). Only the Notes (:310) were amended, so the step contradicts its own code, and the "no event" half of the Done when has no test because it is no longer true. A later reader (7b, feature 8) following the step text would remove the call. **Suggested fix:** Amend the step and its Done when to the agreed rule: no connection makes no call; with a connection the event is removed by the id made from the booking, saved or not, and Google's 404 counts as done. **Resolution:** Fixed 2026-10-03: step 7a.3 and its Done when in current-feature.md now say the agreed rule: removed by the made id, saved or not; no connection makes no call; a connected person with no event gets a DELETE that Google answers 404, done, with no warning; the DELETE carries the person's own key. Closed 2026-10-03 by independent review of step 7a.4: current-feature.md 7a.3 and its Done when (:151-166) state the agreed rule and match remove-booking-event.ts and its tests.

7a/F-125 [P3] closed - cancelledIn is typed `never` where the removal starts, so that call is not type-checked

File: backend/lib/booking/cancel-booking.ts:27, :98 **Found:** 2026-10-03 by independent review of step 7a.3 (scope: 12e7fd3..7df5944; lenses: quality, security, performance, tests) **Why it matters:** `let`

`cancelledIn: string | null = null` is narrowed by TypeScript to `null` at its declaration, and an assignment inside the transaction callback (:90) does not undo that, so inside `if (cancelledIn)` (:98) the variable is `never`. Reproduced in a scratch file with the same shape under `--strict`: assigning it to a `number` compiles. The runtime is right today (a failed commit throws before :98, and only a cancel that changed something sets it), but `start(cancelledIn, ...)` would accept any argument type, so a later change to `start`'s parameters or to what is stored there would compile silently. **Suggested fix:** Return the business from the transaction with the result (for example `{ result, cancelledIn }`) and read it from there, or declare it as `let cancelledIn = null as string | null` so the type is not narrowed away. **Resolution:** Fixed 2026-10-03: `cancelledIn` is declared with a cast (`null as string | null`), so TypeScript keeps its type after the transaction and checks the removal's start. The behaviour is unchanged. Closed 2026-10-03 by independent review of step 7a.4: `cancel-booking.ts:31`; a probe `const probe: number = cancelledIn` inside `if (cancelledIn)` (:103) fails tsc with "Type 'string' is not assignable to type 'number'", so both starts (:104, :105) are type-checked. No new defect.

7a/F-126 [P3] closed - The cancel's DTSTAMP lookup and the "only a cancelled booking" guard are not tested

File: `backend/lib/email/send-cancellation-emails.ts:38-50, :28` (tests: `backend/lib/email/send-cancellation-emails.test.ts:327`) **Found:** 2026-10-03 by independent review of step 7a.4 (scope: `bea19bc..dfa2d60`; lenses: quality, security, performance, tests) **Why it matters:** The spec's contract (Data / contracts, "The withdrawn invite") says the invite is stamped with the moment of the cancel, read from its timeline entry. Two mutations leave all 172 tests in `lib/email`, `lib/booking`, `emails` and `lib/calendar` green: stamping with `context.createdAt` always, and querying `payload->>'booking_id'` (a broken lookup that silently falls back to the booking's own moment). The "same invite again" test (:327) only proves the stamp is stable, which the fallback also is. So the payload JSON query, the one new query in this step, is never shown to find its entry. Separately, removing `if (context.status !== "cancelled") return []` (:28) also stays green: nothing proves a confirmed booking gets no cancellation, which feature 8's retries will rely on (feature 6 has the mirror test, "a cancelled booking sends nothing"). **Suggested fix:** In the "sends both" test, read the `booking_cancelled` entry's `occurredAt` and expect the invite to carry **DTSTAMP**: of that moment (and differ from the booking's `createdAt`, which the test's real clock already makes true). Add "a booking still confirmed sends no cancellation": call `sendCancellationEmails` on an uncanceled booking and expect no `Resend` call. **Resolution:** Fixed 2026-10-03: the "sends both" test makes the booking a day older, then expects the invite's **DTSTAMP** to equal the `booking_cancelled` entry's `occurredAt`; a new test expects a booking still confirmed to get no cancellation and no `Resend` call. Always stamping with the booking's moment, a wrong JSON key in the lookup, and dropping the status check each fail a test. The lookup also filters on the contact, which uses the timeline index. Closed 2026-10-03 by the independent review of step 7a.5, which re-read 33fa2f4: stamping always with the booking's `createdAt`, querying `->>'booking_id'`, and removing the status check each fail exactly one test in `send-cancellation-emails.test.ts` (9 tests); the new `contactId` filter is a non-null inner-joined id, so it narrows without hiding the entry. Nothing new found.

7a/F-127 [P3] closed - `logNothingSent` is a second export in `send-and-record-emails.ts`

File: backend/lib/email/send-and-record-emails.ts:22 **Found:** 2026-10-03 by independent review of step 7a.4 (scope: bea19bc..dfa2d60; lenses: quality, security, performance, tests) **Why it matters:** The backend rule in coding-standards.md (:113-114, Frank, 2026-09-26) is kind, then area, then one file per export, the file named after it. The refactor put two functions in one file: `sendAndRecordEmails` and `logNothingSent`, and a reader importing `logNothingSent` cannot find it by its name. Types beside a function are the codebase's norm; a second function is not. **Suggested fix:** Move `logNothingSent` to its own `backend/lib/email/log-nothing-sent.ts`, or have `findBookingEmailContext`'s two callers share one small helper file named for it. **Resolution:** Fixed 2026-10-03: `logNothingSent` moved to `backend/lib/email/log-nothing-sent.ts`; both senders import it from there. Closed 2026-10-03 by the independent review of step 7a.5: `log-nothing-sent.ts` exports only `logNothingSent`, `send-and-record-emails.ts` exports only `sendAndRecordEmails` beside its two types, and no import of the old location remains.

7a/F-129 [P2] closed - The confirm step is not announced, and opening it drops keyboard focus

File: frontend/components/booking-page/booking-page.tsx:158-167 **Found:** 2026-10-03 by independent review of step 7a.5 (scope: 33fa2f4..78a97a7; lenses: quality, security, performance, tests) **Why it matters:** Step 7a.5 says "Accessible: the confirm step and the result are announced". The result is (the aria-live heading and the `role="alert"` problem), the confirm step is not. Pressing "Cancel this booking" swaps that button for the `role="group"` block, so the focused element is removed and focus falls to the page body; the group is not a live region and nothing moves focus into it. A screen reader user hears nothing after pressing the button and does not learn that "Cancel this booking?" with two new buttons appeared. The same happens on "Keep it", whose button is removed in turn. **Suggested fix:** When the step becomes "confirm", move focus to the question or to "Keep it" (the safe choice), for example with a ref and the same `requestAnimationFrame` the result uses; when "Keep it" is pressed, return focus to "Cancel this booking". Optionally put the question inside the existing polite live region. **Resolution:** Fixed 2026-10-03: opening the confirm step moves focus to "Keep it" and the question is a status region, so it is read out; "Keep it" puts focus back on "Cancel this booking". Checked by the build and lint; the frontend has no test runner, so no saved test. Closed 2026-10-03 by the independent review of step 7a.6: `booking-page.tsx`:171-174 moves focus to "Keep it" (`keepRef`, after the commit through `requestAnimationFrame`), and because "Keep it" sits inside `role="group" aria-labelledby="confirm-cancel"` (:180-181), entering the group reads "Cancel this booking?" with it; "Keep it" returns focus to "Cancel this booking" (`cancelRef`, :197-200). The `role="status"` on the question is redundant with the group label (a live region mounted already filled is not reliably read) but harmless. No new defect from the repair; lint passes.

7a/F-130 [P3] closed - White text on the business's colour has no contrast check

File: frontend/components/booking-page/booking-page.tsx:81, :176, :206 **Found:** 2026-10-03 by independent review of step 7a.5 (scope: 33fa2f4..78a97a7; lenses: quality, security, performance, tests) **Why it matters:** The two main actions ("Yes, cancel it" and "Call <business>") are white text on `brandColor`, and the only rule on that colour is `#rrggbb` (shared validation schema and the database check). A business with a light brand colour, a yellow such as `#facc15`, gets about 1.5:1 contrast, far under WCAG's 4.5:1, so the button that confirms the cancel is close to unreadable; at

`disabled:opacity-60` it is worse. The emails have the same pattern (EmailButton), so this is a choice made once for both rather than a page-only slip. The inline `style` it needs is also the frontend's first, against the "No inline styles" standard, with no comment saying why. **Suggested fix:** A small shared helper that picks white or a dark ink from the colour's relative luminance, used by the page and EmailButton; or fall back to the neutral ink when white on the brand is under 4.5:1. Say in a comment that a per-business colour is the one inline style. **Resolution:** Fixed 2026-10-03: `textColorOn` (packages/shared/helpers/text-color-on.ts, tested) picks white or dark ink by contrast; the page's two buttons and the emails' EmailButton use it. The page's inline styles carry a comment: the business's colour is data, not a class. Closed 2026-10-03 by the independent review of step 7a.6: `text-color-on.ts` computes WCAG relative luminance and returns whichever of white and `#0f172a` contrasts more; its test pins `#facc15` to the ink and `#1d4ed8` to white (shared suite, 107 passing). `booking-page.tsx:87-88, :189, :221` and `email-button.tsx:264` use it; a rendered confirmation on `#facc15` shows `color:#0f172a` on the button. The yellow case of the finding now reads at about 12:1. Residual, not a defect of the repair: with two text colours the worst case is a mid-luminance brand colour (for example `#7a7a7a`) at about 4.3:1, a little under 4.5:1 for 14-16px semibold text; only a third colour or a darkened brand could lift that.

7a/F-131 [P3] closed - "Try again" gives no sign it did anything

File: `frontend/components/booking-page/booking-page.tsx:26-36, :64` **Found:** 2026-10-03 by independent review of step 7a.5 (scope: `33fa2f4..78a97a7`; lenses: quality, security, performance, tests) **Why it matters:** On the unreachable screen, "Try again" bumps `reloads` but leaves `result` as it was, so the screen does not change while the request runs, and if it fails again the identical screen stays. Jane cannot tell a retry happened from a button that does nothing, which is the moment she is most likely to give up and not cancel. **Suggested fix:** Clear the result on retry (`setResult(null)` with the reload), so "One moment..." shows until the new answer arrives. **Resolution:** Fixed 2026-10-03: "Try again" clears the result first, so "One moment..." shows while it retries. Also from the notes: the logo keeps its shape (object-contain). Re-examined 2026-10-03 by the independent review of step 7a.6 and not closed: the visible sign is there (`booking-page.tsx:65-68` clears the result, so "One moment..." shows), but the repair removes the focused "Try again" button from the page and nothing announces the retry or its answer, tracked as F-133. Close both together once F-133 is fixed. Closed 2026-10-03 by the independent review of the whole of feature 7a (`32114fc..14772a1`), together with F-133: `booking-page.tsx:83-87` sets `retried`, clears the result and bumps `reloads`, so "One moment..." (a `role="status"` line, `:48`) shows while the request runs, and the effect at `:41-43` moves focus to the next screen's heading once it is drawn. No new defect in the repair.

7a/F-132 [P3] closed - AGENTS.md's list of where the ports move together leaves out half the places that carry them

File: `AGENTS.md:449-452` **Found:** 2026-10-03 by independent review of step 7a.6 (scope: `8c47953..34cc243`; lenses: quality, security, performance, tests) **Why it matters:** The new paragraph reads as the checklist for the next port move: `PORT`, `BETTER_AUTH_URL` and `APP_ORIGIN` in `.env`, the `-p` in the frontend's scripts, and three code fallbacks. The commit itself also had to change `.claude/launch.json` (`:8, :14`), `.env.example` (`PORT`, `BETTER_AUTH_URL`, `APP_ORIGIN`, `NEXT_PUBLIC_API_URL` and the Google redirect comment), the `APP_ORIGIN` fallback in the three route

tests (public-booking-links-routes.test.ts:34, public-booking-page-routes.test.ts:43, public-bookings-routes.test.ts:41), the sign-in line in packages/shared/scripts/seed-dev.ts:552 and the sample link in backend/scripts/email-preview.ts:35; and a local `frontend/.env.local` with `NEXT_PUBLIC_API_URL` would need it too. Following the list as written leaves launch.json on the old ports, so the preview tool waits on a port nothing listens on. Every place moved correctly this time (`git grep` finds no 3000/3001 left outside that paragraph); only the checklist is short. **Suggested fix:** Name the missing places in the paragraph, or replace the list with the one command that finds them all, `git grep -nE '340[01]'`, plus the two that git cannot see (the local `.env` / `frontend/.env.local` and Google's redirect). **Resolution:** Fixed 2026-10-03: AGENTS.md no longer lists files by hand; it says `git grep -n -E "340[01]"` lists every place in the repo, and names the two outside it (the root `.env`'s `PORT`, `BETTER_AUTH_URL` and `APP_ORIGIN`, and `frontend/.env.local`'s `NEXT_PUBLIC_API_URL` if that file exists). Closed 2026-10-03 by the independent review of the whole of feature 7a (32114fc..14772a1): AGENTS.md:444-454 now points at `git grep -n -E "340[01]"` instead of a hand list, and names the root `.env` lines, `frontend/.env.local` and Google's redirect, which git cannot see. A `git grep` for `localhost:300[01]` and `-p 300` outside `blueprint/` finds nothing left on the old ports. No new defect.

7a/F-133 [P3] closed - "Try again" now drops keyboard focus, and nothing announces the retry or its answer

File: frontend/components/booking-page/booking-page.tsx:39-45, :63-68 **Found:** 2026-10-03 by independent review of step 7a.6 (scope: 8c47953..34cc243; lenses: quality, security, performance, tests) **Why it matters:** F-131's repair calls `setResult(null)` on "Try again", which replaces the whole card with "One moment...". The button that holds focus is unmounted, so focus falls to the page body, and neither "One moment..." nor the screen that follows is a live region or receives focus. A screen reader user who presses "Try again" hears nothing, and if it fails again lands on the same screen with focus nowhere; if it works, the booking appears unannounced. That is F-131's "Jane cannot tell a retry happened" again, for the reader who cannot see the change, and the focus loss is new with the repair. The same page's cancel result already solves this with a polite live region and a focused heading. **Suggested fix:** Put the loading line and the state headings in one `aria-live="polite"` region that stays mounted (for example inside `PageFrame`), or focus the new screen's heading after each answer, as `resultHeading` does for the cancel. **Resolution:** Fixed 2026-10-03: after "Try again", focus moves to the next screen's heading (the "can't load" heading again, or "Your booking"), from an effect that runs once that screen is rendered; "One moment..." is a status region. The first load moves no focus. Checked in the browser with the API stopped then started: focus landed on "Your booking can't load right now", then on "Your booking". The frontend has no test runner, so no saved test; build and lint pass. Closed 2026-10-03 by the independent review of the whole of feature 7a (32114fc..14772a1): the effect at booking-page.tsx:41-43 runs after the commit in which the new screen's heading ref is attached (the not-found and unreachable headings at :57-59 and :73-75, and `BookingDetails`' heading through the ref callback at :172-175), so focus lands on that heading after every retry and never on the first load; the cancel's own focus (:131, :137) is unchanged. Frontend build and lint pass in this pass. No new defect.

Independent review

Status: passed **Target commit:** 14772a1c35391f4e7c8f1c30b16813493d75bec7 **Base commit:** 32114fc4260d126d8b655e322b8cce9dc795f1bd **Base ref:** main **Spec hash:** 4c1d273fc6df508528e9f4e34b1068182d517fd21f044de36581ee9a8bda6465 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-03T17:57:23Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-03T18:03:15Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

32114fc4260d126d8b655e322b8cce9dc795f1bd..14772a1c35391f4e7c8f1c30b16813493d75bec7 delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `git rev-parse HEAD` : pass (equals Target commit) - `git merge-base main HEAD` : pass (equals Base commit) - `sha256sum blueprint/context/current-feature.md` : pass (equals Spec hash) - `git status --short` : pass (only `blueprint/context/review.md` modified, plus the untracked `blueprint/ai-voice-proposal.md` named under Remaining risk) - `npm run test -- workspace=@scheduleleads-app/shared` : pass (14 files, 107 tests) - `npm run test -- workspace=backend` : pass (52 files, 503 tests, against the local seeded `scheduleleads_dev`) - `npm run build --workspace=backend` : pass - `npm run build --workspace=frontend` : pass (typed public client sees both new routes; `/b/[token]` dynamic, `/b/icon.svg` served) - `npm run lint -- workspace=frontend` : pass - `npm run format:check` : pass

Evidence

- Token (backend/lib/booking/booking-page-token.ts): HMAC-SHA256 over `booking-page:<id>` under a 32-byte key read through the shared `readBase64Key`; strict UUID and 43-char base64url shape, canonical spelling only, `timingSafeEqual`; the API refuses to start without the key (backend/server.ts). - Routes (backend/routes/public-booking-page-routes.ts): mounted under `/public/` with the public CORS rule (no credentials, APP_ORIGIN allowed); `Cache-Control: no-store` on both; every bad link answers the same `refuse("not_found") 404`; cancel is POST only, so mail link scanners that prefetch the GET cancel nothing; no path collides with `/:slug/booking-links` or `POST /:slug/bookings`. - Cancel (backend/lib/booking/cancel-booking.ts): one transaction, booking row locked for update before deciding, update guarded by `status = 'confirmed'`, held rows of that booking only released, one `booking_cancelled` entry with `{ bookingId }` and a null actor; Google removal and emails start only when this call changed the row (`cancelledIn`), so a second press does neither. - Fit between steps: removal and emails take the business from the cancel's own row; both re-check `status = 'cancelled'`; the confirmation sender re-checks `confirmed`, so a late confirmation never follows a cancel; the CANCEL invite reuses the booking UID with SEQUENCE 1 and a DTSTAMP read from the cancel's timeline entry, so a retry is byte-identical under the same idempotency key (tested). - Data

exposure: the page view (find-booking-page.ts) selects no contact or lead column; route tests assert none of the customer's details in either answer; log lines carry ids and reasons only (tested in remove-booking-event.test.ts and send-cancellation-emails.test.ts). - Frontend: `/b/` sends `Referrer-Policy: no-referrer`, `no-store` and `X-Robots-Tag` (next.config.ts) and the layout sets `noindex` and `referrer`; the root title is overridden and `/b/icon.svg` replaces the root icon; calls go through the public client only; business website is validated `https://` at write time and React 19 blocks `javascript:` hrefs. - Migration 0017 only widens `activity_type_check` with `booking_cancelled`, matching `ACTIVITY_TYPES`. - Port chore: no `localhost:3000/3001` or `-p 300x` left outside `blueprint/`; AGENTS.md, `.env.example`, launch.json, fallbacks and seed line moved together. - Re-examined F-131, F-132, F-133 (fixed) against the target and closed them in the ledger. - No skipped, focused or placeholder tests in the delta's test files.

Findings

- F-134 [P3] open: the cancel test that checks log lines for the customer's details reads them before the after-cancel work writes any (backend/lib/booking/cancel-booking.test.ts:299-311) - Closed this pass: F-131, F-132, F-133 - No P0 or P1 open or fixed

Remaining risk

- Untracked path outside the target: `blueprint/ai-voice-proposal.md`, the project owner's personal note, never committed and unrelated to this feature; not read or touched by this review. - The customer's page has no saved test: the frontend has no test runner and the project has no browser test harness; the page was reviewed as code only, and its hand check in the browser is the builder's (spec 7a.5). No dev server was started in this review. - `npm run email:preview` was not run here (its check is by hand, spec 7a.4 and 7a.6). - F-128 (unverified): no real calendar has yet been shown to remove the event from the attached CANCEL invite. - Known gap named in the spec: a cancel in the first moment after booking, whose Google write finishes after the removal, leaves the event in place until feature 8's retries. - No security scanner, dependency audit, `Verify` command or GitHub check is declared; none was run. - Dashboard activity (`blueprint/.state/run.json`) was not written by this reviewer, which was limited to the findings ledger and this receipt.